

Advanced Topics in Digital Image Processing

LESSON 10 - DNN + Optical Flow

Objective:

In this class students should familiarize themselves with the algorithms of Deep Neural Networks, Optical Flow and Tensor Flow and evaluate some of their potentialities for image processing.

Procedure:

1 - DNN – Apply the face recognition DNN model (*opencv_face_detector_uint8.pb*) and show the detected face (rectangle) over the image or webcam. (functions: *readNetFromTensorflow*, *blobFromImage*, etc.) (see next page)

2 – Optical Flow – Using the functions in the *ImageOpticalFlow.py* file, test the optical flow methods Lucas-Kanade and Farneback on a video from a webcam or from a video file (pedestrians).

Methods on *imageOpticalFlow.py*:

- *LucasKanade_OF(videoPath, threshold)* - use threshold to filter small movements (>5)
- *Farneback_OF(video_path)*

3 – Tensor Flow – Create and save a DNN using the simple architecture presented in the DNN slides and test it with OpenCV. The result of OpenCV's *net.forward()* for the MNIST digits dataset is an array of size 10 representing each of the 0..9 digits.

Note: to use TensorFlow you need to install TensorFlow using pip or package install from PyCharm.

Pip -install tensorflow

DNN for face detector

```
def getFaceBox( frame, conf_threshold=0.7):
    frameOpencvDnn = frame.copy()
    frameHeight = frameOpencvDnn.shape[0]
    frameWidth = frameOpencvDnn.shape[1]

    # Open DNN model
    modelFile = "data\\opencv_face_detector_uint8.pb"
    configFile = "data\\opencv_face_detector.pbtxt"
    net = cv2.dnn.readNetFromTensorflow(modelFile, configFile)
    # prepare for DNN
    blob = cv2.dnn.blobFromImage(frameOpencvDnn,
                                  scalefactor=1.0,
                                  size=(500, 500),
                                  mean= [104, 117, 123],
                                  swapRB=False,
                                  crop=False)

    #set image as DNN input
    net.setInput(blob)

    #get Output
    detections = net.forward()

    bboxes = []
    for i in range(detections.shape[2]):
        confidence = detections[0, 0, i, 2]
        if confidence > conf_threshold:
            x1 = int(detections[0, 0, i, 3] * frameWidth)
            y1 = int(detections[0, 0, i, 4] * frameHeight)
            x2 = int(detections[0, 0, i, 5] * frameWidth)
            y2 = int(detections[0, 0, i, 6] * frameHeight)
            bboxes.append([x1, y1, x2, y2])
            cv2.rectangle(frameOpencvDnn, (x1, y1), (x2, y2), (0, 255, 0),
                          int(round(frameHeight/150)), 8)

    return frameOpencvDnn, bboxes
```

Returns an image marked with rectangles and a list of detected faces.